الـ Abstract Class بيساعدك:

تبني Architecture نظيف

تطبق مبدأ "Code against abstraction"

تقلل التكرار

تخلي الكود قابل للتوسعة بدون تعديل في الكود القديم

لو بتشتغل بـ OOP وخصوصًا في C#، استخدام الـ Abstract Class في المكان الصح بيخلي تصميمك أقوى بكتير 💪

#CSharp #DotNet #OOP #BackendDevolper #SoftwareEngineering #Programming

11:12

```
  }
}
```

كده أي Shape لازم يبقى عنده طريقة لحساب المساحة،
لكن كل شكل بيحسبها بطريقته الخاصة.

الفرق بين Abstract Class و Interface

الـ Abstract Class ممكن يحتوي على:

Methods عادية

Abstract Methods

Fields

Constructors

الـ Interface بيركز أكتر على "إيه المطلوب يتنفذ" بدون تفاصيل تنفيذ.

الخلاصة 💡

Add a comment...

11:12

Shape مفيهاش تنفيذ → لازم أي كلاس يرث من GetArea() ينفذها.

فيها تنفيذ عادي → الكلاسات الوريثة تقدر تستخدمها زي ما Print() هي.

ليه نستخدم Abstract Class؟

✅ لما يكون عندنا كذا كلاس بيشتركوا في سلوك عام
✅ لما نحب نفرض قواعد معينة على الكلاسات اللي بترث
✅ لما نطبق مبدأ Abstraction بشكل عملي

مثال عملي

```
public class Circle : Shape
{
   public double Radius { get; set; }

   public override double GetArea()
   {
      return Math.PI * Radius * Radius;
   }
```

Add a comment...

11:11

What is an Abstract Class in OOP? 🤔

في عالم البرمجة الكائنية (OOP)، من أهم المفاهيم اللي بتساعدنا نكتب كود منظم وقابل للتوسّع هو مفهوم Abstract Class.

أولًا: يعني إيه Abstract Class؟

الـ Abstract Class هو كلاس بنستخدمه كـ "قالب" أو "أساس" لكلاسات تانية،
لكن ما ينفعش نعمل منه Object بشكل مباشر.

بمعنى تاني 👇
هو بيحدد الفكرة العامة، لكن بيسيب التفاصيل للتنفيذ في الكلاسات اللي بترث منه.

في C# بنستخدم كلمة abstract:

```
public abstract class Shape
{
   public abstract double GetArea();

   public void Print()
   {
      Console.WriteLine("This is a shape");
   }
}
```

هنا:

Add a comment...

11:12

لو بتشتغل بـ OOP وخصوصًا في C#، استخدام الـ Abstract Class في المكان الصح بيخلي تصميمك أقوى بكتير 💪

#CSharp #DotNet #OOP #BackendDevolper #SoftwareEngineering #Programming

Show translation

Like Comment Repost Send

2 impressions View

Add a comment...

OOP Concepts – Interfaces & Abstraction

2- What do we mean by coding against interface rather than class?

Coding against an interface means writing your code to depend on an interface or abstract class instead of a specific concrete class. The code focuses on what the object should do, not how it does it.

This approach increases flexibility and reduces tight coupling between components.

Coding against abstraction, not concreteness

Abstraction refers to interfaces or abstract classes that define behavior without implementation details. Concreteness refers to concrete classes that contain the actual implementation.

Coding against abstraction means that your code depends on abstractions (interfaces or abstract classes) rather than concrete implementations, making the system easier to extend, maintain, and test.

3- What is abstraction as a guideline and how can we implement it?

Abstraction is a design guideline that focuses on hiding complex implementation details and exposing only essential functionality to the user.

It encourages developers to:
- Focus on what an object does, not how it does it
- Expose only necessary operations
- Hide internal complexity

How abstraction is implemented in C#:

1. Interfaces: Define what a class must do without specifying how it does it.
2. Abstract Classes: Provide partial implementation while forcing derived classes to complete the behavior.
3. Access Modifiers: Use private members to hide details and public members to expose required behavior.